

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN**



HCMUTE

**BÁO CÁO ĐỒ ÁN MÔN HỌC
THỊ GIÁC MÁY TÍNH**

ĐỀ TÀI:

**TÌM HIỂU KỸ THUẬT GRAPH-BASE SLAM
TỪ MONOCULAR VISUAL ODOMETRY
ĐẾN XÂY DỰNG POSE GRAPH VỚI GTSAM**

GVHD: PGS.TS: Hoàng Văn Dũng

Học viên thực hiện:

2611318 - Vòng Vĩnh Shìn

2611302 - Trần Nhật Duật

TP. Hồ Chí Minh, tháng 07 năm 2026

THÔNG TIN NHÓM VÀ TỶ LỆ ĐÓNG GÓP

STT	MSHV	Họ và tên	Tỷ lệ đóng góp (%)	Ghi chú
1	2611318	Vòng Vĩnh Shìn	100%	
2	2611302	Trần Nhật Duật	100%	

BẢNG PHÂN CÔNG NHIỆM VỤ

STT	Họ và tên	Nhiệm vụ được phân công	Kết quả
1	Trần Nhật Duật	Tìm hiểu SLAM và Visual Odometry trình bày ORB, Essential Matrix, RANSAC và scale ambiguity, biên soạn cơ sở lý thuyết liên quan.	Hoàn thành
2	Vòng Vĩnh Shìn	Cài đặt khối tiếp nhận – tiền xử lý video và phát hiện/so khớp đặc trưng ORB, tham gia thực nghiệm, thu thập và phân tích số liệu so khớp.	Hoàn thành
3	Vòng Vĩnh Shìn	Nghiên cứu Pose Graph, factor graph, mô hình nhiễu và nguyên lý tối ưu Pose Graph, biên soạn phần lý thuyết liên quan.	Hoàn thành
4	Trần Nhật Duật	Cài đặt khối Visual Odometry, xây dựng Pose Graph bằng GTSAM, trực quan hóa và lưu kết quả; tham gia thực nghiệm và phân tích quỹ đạo.	Hoàn thành

MỤC LỤC

CHƯƠNG 1: MỞ ĐẦU.....	7
1.1. Lý do chọn đề tài	7
1.2. Mục tiêu đề tài	8
1.3. Đối tượng và phạm vi nghiên cứu	8
1.4. Nội dung thực hiện	8
1.5. Phương pháp thực hiện	9
1.6. Dự kiến kết quả.....	9
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	11
2.1. Tổng quan về SLAM	11
2.2. Phân loại các phương pháp SLAM.....	11
2.2.1. SLAM dựa trên bộ lọc (Filter-based SLAM)	11
2.2.2. SLAM dựa trên đồ thị (Graph-based SLAM)	11
2.2.3. So sánh Visual SLAM và LiDAR SLAM	11
2.3. Pose Graph và phương pháp Graph-based SLAM	12
2.4. Monocular Visual Odometry: Essential Matrix, RANSAC và Scale Ambiguity	14
CHƯƠNG 3: GIẢI PHÁP	16
3.1. Một số kiến trúc tham khảo	16
3.2. Sơ đồ tổng quát giải pháp	17
3.2.1. Khối tiếp nhận, tiền xử lý video và ước lượng ma trận K.....	18
3.2.2. Khối trích xuất đặc trưng ORB	19
3.2.3. Khối so khớp đặc trưng bằng k-NN	19
3.2.4. Khối Visual Odometry	19
3.2.5. Khối xây dựng Pose Graph.....	20

3.2.6. Khởi render & lưu kết quả.....	20
3.3. Thư viện sử dụng và môi trường cài đặt.....	20
3.3.1. Thư viện sử dụng	20
3.3.2. Môi trường cài đặt	21
3.4. Thực nghiệm.....	22
3.4.1. Môi trường và thiết lập thực nghiệm.....	22
3.4.2. Quá trình thực hiện qua từng bước	23
3.4.3. Kết quả thực nghiệm	29
3.5. Phân tích và đánh giá kết quả	32
CHƯƠNG 4: KẾT LUẬN.....	33
4.1. Kết luận.....	33
4.2. Hạn chế.....	33
4.3. Hướng phát triển.....	34
TÀI LIỆU THAM KHẢO	35

DANH MỤC BẢNG

Bảng 1: So sánh Visual SLAM và SLAM dựa trên LiDAR	12
Bảng 2: So sánh ba loại kiến trúc SLAM phổ biến và giải pháp trong đề tài	17
Bảng 3: Danh sách thư viện sử dụng trong pipeline	21
Bảng 4: Thông số cấu hình thực nghiệm.....	23
Bảng 5: Kết quả định lượng thực nghiệm	32

DANH MỤC HÌNH ẢNH

Hình 1: Minh hoạ một Pose Graph với 6 Pose	13
Hình 2: Hình học Epipolar giữa hai khung	14
Hình 3: Kiến trúc pipeline Visual Odometry/Pose Graph được cài đặt trong đề tài.....	18
Hình 4: Khung hình gốc, sau resize và ảnh grayscale (kèm kích thước từng bước).....	24
Hình 5: Keypoint ORB trên khung hình đầu tiên.....	25
Hình 6: Giao diện Bước 3: mẫu good match k-NN sau Lowe ratio test giữa hai khung hình đầu chuỗi	26
Hình 7: Thống kê Visual Odometry và match sau RANSAC ở cặp inlier (xanh), outlier (đỏ).....	27
Hình 8: Giao diện tab Pose Graph (GTSAM) số node/factor, sơ đồ factor graph với bốn pose đầu $X(0)$ – $X(3)$ và BetweenFactor giữa các pose kế tiếp.	28
Hình 9: Giao diện đồ thị (NetworkX) cận cảnh cấu trúc Pose Graph thực tế, mỗi cạnh kèm chuyển động tương đối (R, t).....	29
Hình 10: Phân bố inlier ratio trên 422 cặp khung hình liên tiếp	30
Hình 11: Inlier ratio theo từng cặp khung hình; các điểm đỏ là cặp rơi vào nhánh fallback	30
Hình 12: Pose Graph thực tế sau khi dựng và render (423 node, 422 cạnh), cạnh tô màu theo inlier ratio.....	31

CHƯƠNG 1: MỞ ĐẦU

1.1. Lý do chọn đề tài

Định vị và lập bản đồ đồng thời (SLAM Simultaneous Localization and Mapping) là một trong những bài toán nền tảng của thị giác máy tính và robot, cho phép một hệ thống (robot, xe tự hành, thiết bị bay không người lái, hoặc đơn giản là một camera cầm tay) ước lượng chuyển động của chính nó xây dựng và biểu diễn không gian xung quanh chỉ từ dữ liệu hình ảnh / cảm biến, không cần hạ tầng định vị bên ngoài như GPS. Một trong những dạng đơn giản và phổ biến nhất của bài toán này là Visual Odometry (VO), chỉ dùng một camera thông thường để ước lượng chuyển động tương đối giữa các khung hình liên tiếp.

Nhóm chúng em lựa chọn đề tài “Tìm hiểu kỹ thuật Graph-based SLAM từ Monocular Visual Odometry đến xây dựng Pose Graph với GTSAM” xuất phát từ ba lý do chính sau.

- Thứ nhất, SLAM là chủ đề có tính ứng dụng cao nhưng cũng đòi hỏi một số kiến thức nền tảng. Việc tự cài đặt một pipeline VO/Pose-Graph hoàn chỉnh, dù ở quy mô đơn giản, là cách hiệu quả để nhóm chúng em nắm vững các khái niệm này thay vì chỉ đọc lý thuyết.
- Thứ hai, biểu diễn bài toán dưới dạng Pose Graph là nền tảng chung của hầu hết các SLAM hiện đại, hiểu rõ cách xây dựng đồ thị này từ node, factor là bước chuẩn bị cần thiết trước khi tiếp cận các hệ thống SLAM hoàn chỉnh.
- Thứ ba, nhóm mong muốn làm việc với dữ liệu thực tế video quay bằng điện thoại, camera không qua hiệu chỉnh để trực tiếp quan sát và phân tích những vấn đề thực tế của Monocular visual SLAM.

Xuất phát từ các lý do trên, đề tài nhóm chúng em tập trung xây dựng và phân tích một pipeline Monocular visual SLAM dạng Pose-Graph, đi từ dữ liệu video thô đến việc dựng và trực quan hoá Pose Graph bằng thư viện GTSAM, đồng thời đánh giá trung thực những gì một pipeline ở mức Visual Odometry (chưa có loop closure, chưa tối ưu) có thể thực hiện được những gì.

1.2. Mục tiêu đề tài

Đề tài hướng đến các mục tiêu cụ thể sau:

- Hệ thống hoá cơ sở lý thuyết về SLAM nói chung, trọng tâm tìm hiểu, xây dựng Pose Graph SLAM và bài toán Visual Odometry (Essential Matrix, RANSAC, Scale Ambiguity).
- Cài đặt một pipeline xử lý hoàn chỉnh từ video đầu vào, trích xuất khung hình, phát hiện và so khớp đặc trưng ORB, ước lượng chuyển động, và xây dựng Pose Graph bằng GTSAM.
- Trực quan hoá Pose Graph để xây dựng được quỹ đạo và cấu trúc đồ thị node, cạnh. Giúp quan sát trực tiếp và độ tin cậy của từng ràng buộc Odometry.
- Thực nghiệm trên dữ liệu video thực tế quay bằng điện thoại tại khuôn viên Trường Đại học Công nghệ Kỹ thuật TP. HCM, thu thập số liệu định lượng là số khung hình, số match ORB, Inlier Ratio trung bình, số node/factor trong đồ thị và đánh giá trung thực các hạn chế của pipeline mà không hiệu chỉnh camera, không loop closure, không tối ưu hoá.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là bài toán Monocular Visual Odometry và biểu diễn Pose Graph (Graph-based SLAM) bằng thư viện factor graph GTSAM.

- Phạm vi lý thuyết: mô hình hoá bài toán SLAM dưới dạng Pose Graph. Cấu trúc Pose Graph, hàm sai số và mô hình nhiễu, hình học giữa hai khung hình (essential matrix, RANSAC, phân rã pose, scale ambiguity).
- Phạm vi thực nghiệm: cài đặt và chạy pipeline trên video quay thực tế bằng camera điện thoại (không hiệu chỉnh), giới hạn ở việc dựng Pose Graph từ ràng buộc Odometry giữa các khung hình liên tiếp.
- Đề tài không triển khai: phát hiện & xác định vị trí đã đi qua, bước tối ưu hoá Pose Graph, ước lượng chuyển động vật thể động, hay xây dựng bản đồ 3D.

1.4. Nội dung thực hiện

-
- Khảo sát tài liệu, bài báo khoa học về SLAM, Pose Graph Optimization và Monocular Visual Odometry.
 - Xây dựng cơ sở lý thuyết, mô hình hoá bài toán SLAM, biểu diễn Pose Graph.
 - Thiết kế pipeline xử lý, trích xuất khung hình & tiền xử lý ảnh, front-end ORB, visual odometry, xây dựng Pose Graph GTSAM, render & lưu kết quả.
 - Cài đặt bằng Python (OpenCV, GTSAM, NetworkX) trên Jupyter Notebook; thực nghiệm trên video quay thực tế; thu thập số liệu định lượng.
 - Phân tích, đánh giá kết quả đạt được, hạn chế và đề xuất hướng phát triển.

1.5. Phương pháp thực hiện

Đề tài nhóm chúng em kết hợp phương pháp tìm hiểu lý thuyết và phương pháp thực nghiệm xây dựng ứng dụng.

- Về mặt lý thuyết, nhóm thực hiện tổng hợp và đọc một số công trình, bài báo khoa học đã công bố (chủ yếu là một số bài báo open access trên arXiv và các tạp chí/hội nghị uy tín trong lĩnh vực robotics) để xây dựng nền tảng lý luận nhất quán cho bài toán Pose Graph SLAM và ước lượng chuyển động vật thể.
- Về mặt thực nghiệm, nhóm cài đặt trực tiếp pipeline xử lý ảnh và dựng Pose Graph bằng Python (OpenCV cho thị giác máy tính, GTSAM cho factor graph, NetworkX cho biểu diễn/trực quan hoá đồ thị), chạy trên dữ liệu video quay thực tế bằng điện thoại, sau đó thu thập số liệu định lượng và đối chiếu với cơ sở lý thuyết đã trình bày.

1.6. Dự kiến kết quả

Sau khi hoàn thành, đề tài dự kiến đạt được các kết quả sau:

- Báo cáo trình bày có hệ thống cơ sở lý thuyết về SLAM, Pose Graph và Monocular visual Odometry, có trích dẫn nguồn tham khảo khoa học rõ ràng.
- Một pipeline hoàn chỉnh, có thể chạy lại được, xử lý video thực tế thành Pose Graph và trực quan hoá kết quả đạt được.

- Số liệu kết quả thực nghiệm thực tế mà nhóm thực hiện được (không phải số liệu minh họa) bao gồm như số khung hình xử lý, số match ORB, inlier ratio trung bình, số node/factor trong Pose Graph.
- Nhận xét, đánh giá trung thực về ưu điểm và hạn chế của một pipeline SLAM chỉ ở mức Odometry (chưa thực hiện loop closure và chưa tối ưu hoá). Cuối cùng là đề xuất hướng phát triển tiếp theo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về SLAM

SLAM (Simultaneous Localization and Mapping) là bài toán một robot vừa phải ước lượng vị trí, hướng của chính nó (localization) vừa phải xây dựng bản đồ môi trường xung quanh (mapping), trong khi chỉ có dữ liệu cảm biến (camera, LiDAR, IMU, v.v.) thu được trong quá trình di chuyển, không có bản đồ hay hệ thống định vị toàn cục nào cho trước [1]. Đây là bài toán kinh điển trong robotics. Để định vị chính xác cần có bản đồ, nhưng để xây dựng bản đồ lại cần biết chính xác vị trí robot tại từng thời điểm quan sát.

2.2. Phân loại các phương pháp SLAM

2.2.1. SLAM dựa trên bộ lọc (*Filter-based SLAM*)

Phương pháp này duy trì một ước lượng trạng thái tức thời và cập nhật tuần tự theo thời gian mỗi khi có phép đo mới, không lưu lại toàn bộ lịch sử pose. Ưu điểm là chi phí tính toán cho mỗi bước cập nhật thấp, phù hợp với hệ thống thời gian thực có tài nguyên hạn chế; nhược điểm là sai số tích lũy theo thời gian và khó xử lý bài toán đóng vòng lặp (loop closure) một cách nhất quán toàn cục [1].

2.2.2. SLAM dựa trên đồ thị (*Graph-based SLAM*)

Khác với phương pháp lọc, Graph-based SLAM biểu diễn toàn bộ lịch sử tư thế robot và các ràng buộc giữa chúng dưới dạng một đồ thị, sau đó giải bài toán tối ưu hoá toàn cục trên đồ thị này để có được lời giải nhất quán nhất. Đây là hướng tiếp cận chiếm ưu thế trong các hệ thống SLAM hiện đại như ORB-SLAM2/3 nhờ khả năng tận dụng cấu trúc sparsity của đồ thị và tích hợp hiệu quả với cơ chế đóng vòng lặp [2], [3]

2.2.3. So sánh Visual SLAM và LiDAR SLAM

Bên cạnh cách phân loại theo bộ lọc/đồ thị, một cách phân loại phổ biến khác dựa trên loại cảm biến chính được sử dụng, gồm hai nhóm chủ đạo là Visual SLAM và LiDAR SLAM, mỗi nhóm có những ưu, nhược điểm riêng phù hợp với các kịch bản ứng dụng khác nhau [4], [5]. Dưới đây là tóm tắt so sánh hai nhóm phương pháp này.

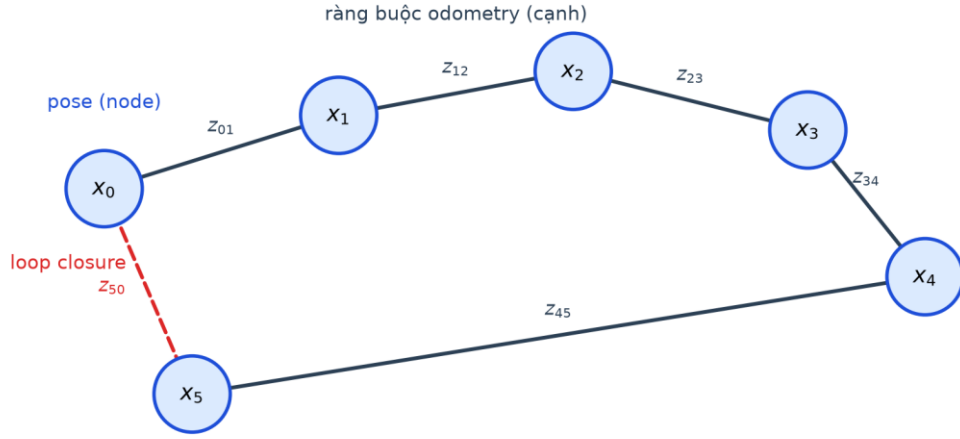
Tiêu chí	Visual SLAM	LiDAR SLAM
Cảm biến	Camera đơn/stereo/RGB-D, chi phí thấp, nhẹ	Cảm biến LiDAR 2D/3D, chi phí và khối lượng cao hơn
Chất lượng dữ liệu	Giàu thông tin ngữ nghĩa và màu sắc, nhạy với ánh sáng	Đo khoảng cách trực tiếp, chính xác hình học cao, ít nhạy ánh sáng
Độ phức tạp tính toán	Cao do phải xử lý ảnh và trích xuất đặc trưng thị giác	Trung bình, chủ yếu xử lý đám mây điểm
Khả năng nhận diện vật thể động	Thuận lợi nhờ kết hợp được nhận dạng/phân đoạn ngữ nghĩa từ ảnh	Hạn chế hơn nếu không kết hợp cảm biến phụ trợ
Ứng dụng tiêu biểu	Robot trong nhà, thiết bị bay, thực tế tăng cường	Xe tự hành ngoài trời, robot công nghiệp phạm vi lớn

Bảng 1: So sánh Visual SLAM và SLAM dựa trên LiDAR

Trong phạm vi đề tài này, do trọng tâm là nghiên cứu bài toán ước lượng chuyển động vật thể. Nhóm lựa chọn tiếp cận theo hướng Visual SLAM/Graph-based SLAM làm nền tảng lý thuyết chính, đồng thời giữ nguyên tính tổng quát của mô hình Pose Graph để có thể mở rộng sang dữ liệu LiDAR trong tương lai nếu cần.

2.3. Pose Graph và phương pháp Graph-based SLAM

Pose Graph là một đồ thị vô hướng $G = (V, E)$ trong đó mỗi đỉnh (node) $v_i \in V$ biểu diễn một tư thế của robot tại thời điểm i , còn mỗi cạnh $e_{ij} \in E$ biểu diễn một ràng buộc chuyển động tương đối giữa hai tư thế v_i và v_j , thu được từ phép đo odometry giữa hai pose liên tiếp hoặc từ việc phát hiện đóng vòng lặp loop closure giữa hai pose không liên tiếp về thời gian nhưng quan sát cùng một vùng không gian [2].



Hình 1: Minh hoạ một Pose Graph với 6 Pose

Mỗi cạnh e_{ij} mang theo một hàm sai số $e_{ij}(x_i, x_j)$ đo mức độ không nhất quán giữa phép đo tương đối z_{ij} và phép đo dự đoán từ ước lượng tư thể hiện tại, cùng ma trận thông tin Ω_{ij} thể hiện độ tin cậy của phép đo. Bài toán tối ưu hoá Pose Graph được thể hiện như sau:

$$x^* = \arg \min_x \sum_{(i,j) \in E} e_{(ij)}^T(x_i, x_j) \Omega_{ij} e_{(ij)}(x_i, x_j)$$

Đây là một bài toán tối ưu phi tuyến không lồi do các thành phần rotation nằm trên manifold chứ không phải không gian Euclid phẳng. Trong thực tế, bài toán được giải lập bằng các phương pháp tuyến tính hoá cục bộ như Gauss-Newton hoặc Levenberg-Marquardt, tận dụng tính thưa của ma trận Hessian để giải hệ phương trình tuyến tính một cách hiệu quả thông qua phân tích Cholesky thưa [2]. Hai thư viện tối ưu hoá đồ thị phổ biến nhất hiện nay là g2o và GTSAM, đều cài đặt các thuật toán nói trên và được sử dụng rộng rãi làm back-end cho các hệ thống SLAM mã nguồn mở.

Trong một hệ thống SLAM đầy đủ, Pose Graph thường không chỉ chứa các đỉnh tư thể robot mà còn chứa các đỉnh điểm mốc quan sát được, tạo thành cấu trúc tổng quát hơn gọi là factor graph, trong đó mỗi factor biểu diễn một ràng buộc xác suất giữa các biến trạng thái [2]. Việc bổ sung điểm mốc giúp gắn kết các tư thể quan sát cùng một vùng không gian, góp phần quan trọng vào việc phát hiện và đóng vòng lặp, giảm sai số tích lũy theo thời gian.

2.4. Monocular Visual Odometry: Essential Matrix, RANSAC và Scale

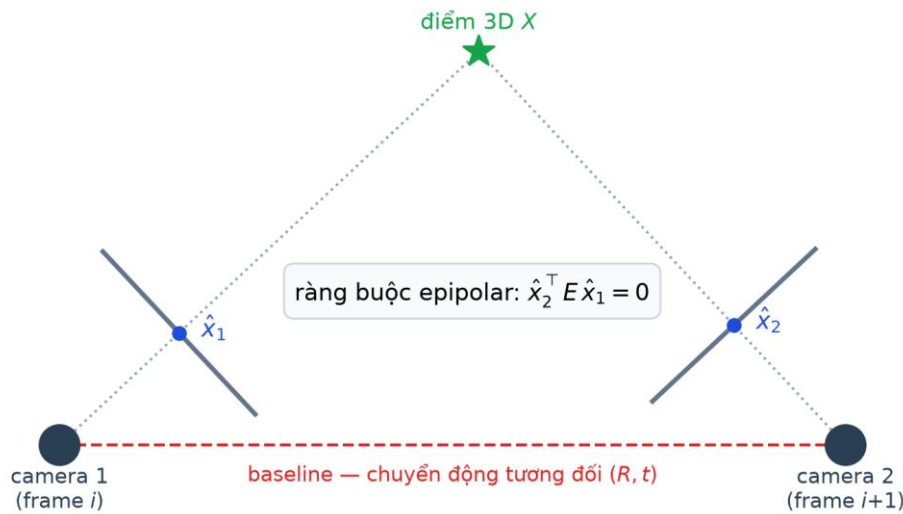
Ambiguity

Visual Odometry (VO) là bài toán ước lượng chuyển động tương đối của camera giữa hai khung hình liên tiếp, chỉ dựa trên các điểm đặc trưng ảnh đã so khớp, không cần bản đồ hay ràng buộc toàn cục. Đây chính là nguồn tạo ra các ràng buộc odometry cho Pose Graph.

Cơ sở hình học của VO là ràng buộc epipolar. Khi cùng một điểm 3D X được quan sát từ hai vị trí camera, hai điểm chiếu tương ứng đã chuẩn hóa $\hat{x}_1, \hat{x}_2$ không thể nằm tùy ý mà phải thỏa:

$$\hat{x}_2^T E \hat{x}_1 = 0, \quad E = [t]_{\times} R$$

trong đó E là ma trận Essential, chứa trọn thông tin chuyển động tương đối (R, t) giữa hai camera qua công thức $E = [t]_{\times} R$ (với $[t]_{\times}$ là ma trận đối xứng lệch của t). Nói cách khác, nếu tìm được E từ các cặp điểm khớp, ta “giải nén” được chuyển động camera từ nó.



Hình 2: Hình học Epipolar giữa hai khung

Thực tế các cặp điểm khớp luôn lẫn sai outlier, nên E được ước lượng bằng thuật toán 5-điểm bậc trong RANSAC, mỗi vòng lấy ngẫu nhiên một tập điểm tối thiểu, ước lượng E , rồi đếm số điểm đồng thuận mô hình có nhiều điểm đồng thuận nhất được giữ

lại. Tỷ lệ điểm đồng thuận trên tổng số điểm khớp inlier ratio trở thành chỉ báo độ tin cậy của phép ước lượng, và là đầu vào trực tiếp cho mô hình nhiều của Pose Graph.

Từ E, phép phân rã giá trị kỳ dị (SVD) cho ra bốn tổ hợp nghiệm (R , t) khả dĩ; nghiệm đúng được chọn bằng điều kiện chirality điểm 3D tam giác hoá phải nằm phía trước cả hai camera. Riêng với camera đơn, t khôi phục từ E chỉ xác định được hướng chứ không xác định được độ lớn thật, cùng một chuyển động ảnh có thể do camera dịch 1 mét trong cảnh lớn hoặc 10 cm trong cảnh nhỏ. Đây là scale ambiguity bất định tỷ lệ cố hữu của VO đơn mắt, chỉ giải quyết được khi có nguồn thông tin tỷ lệ ngoài. Khi không có, cách xử lý phổ biến được áp dụng trong đề tài của nhóm là chuẩn hoá mỗi bước tịnh tiến về độ dài đơn vị, hình dạng quỹ đạo được giữ nguyên, nhưng khoảng cách chỉ mang tính tương đối.

Cuối cùng, pose toàn cục được suy ra bằng cách nối chuỗi các chuyển động tương đối liên tiếp:

$$R_{i+1} = R_i \cdot R, \quad t_{i+1} = t_i + R_i \cdot \hat{t}$$

Phép nối chuỗi này chính là nguồn gốc của drift sai số nhỏ ở mỗi bước không biến mất mà bị mang theo và khuếch đại vào mọi bước sau, vì t_{i+1} phụ thuộc đệ quy vào toàn bộ lịch sử $R_0, \dots, R_i$. Không có loop closure, drift chỉ tăng chứ không giảm đây là lý do mục 2.4 nhấn mạnh vai trò của cạnh loop closure trong việc sửa sai số tích lũy.

CHƯƠNG 3: GIẢI PHÁP

3.1. Một số kiến trúc tham khảo

Trước khi lựa chọn kiến trúc cài đặt, nhóm đã khảo sát ba nhóm kiến trúc SLAM phổ biến Visual SLAM, Visual-Inertial SLAM và LiDAR SLAM tổng hợp trong Bảng 3.1. Việc so sánh giúp làm rõ vị trí của giải pháp cài đặt trong đề tài một pipeline VO/pose-graph tối giản dùng camera đơn, tập trung vào front-end và biểu diễn đồ thị so với các nhóm hệ thống SLAM hoàn chỉnh.

Loại SLAM	Cảm biến	Ưu điểm	Nhược điểm
Visual SLAM	Camera đơn / stereo / RGB-D	Chi phí thấp, nhỏ gọn; ảnh giàu thông tin kết cấu và màu sắc, thuận lợi cho nhận dạng nơi chốn (place recognition)	Nhạy với ánh sáng, motion blur và cảnh ít kết cấu; camera đơn không xác định được scale metric
Visual-Inertial SLAM	Camera + IMU	IMU bù chuyển động nhanh và mờ ảnh, khôi phục được scale metric; bền vững hơn khi đặc trưng thị giác kém trong thời gian ngắn	Phải đồng bộ và hiệu chỉnh camera-IMU phức tạp; IMU trôi nhanh nếu thiếu quan sát thị giác bổ trợ
LiDAR SLAM	LiDAR 2D/3D (thường kết hợp IMU)	Đo khoảng cách trực tiếp, độ chính xác hình học cao, ổn định trước thay đổi ánh sáng	Cảm biến đắt và nặng; điểm đám mây thừa thông tin ngoại hình nên nhận dạng nơi chốn khó hơn

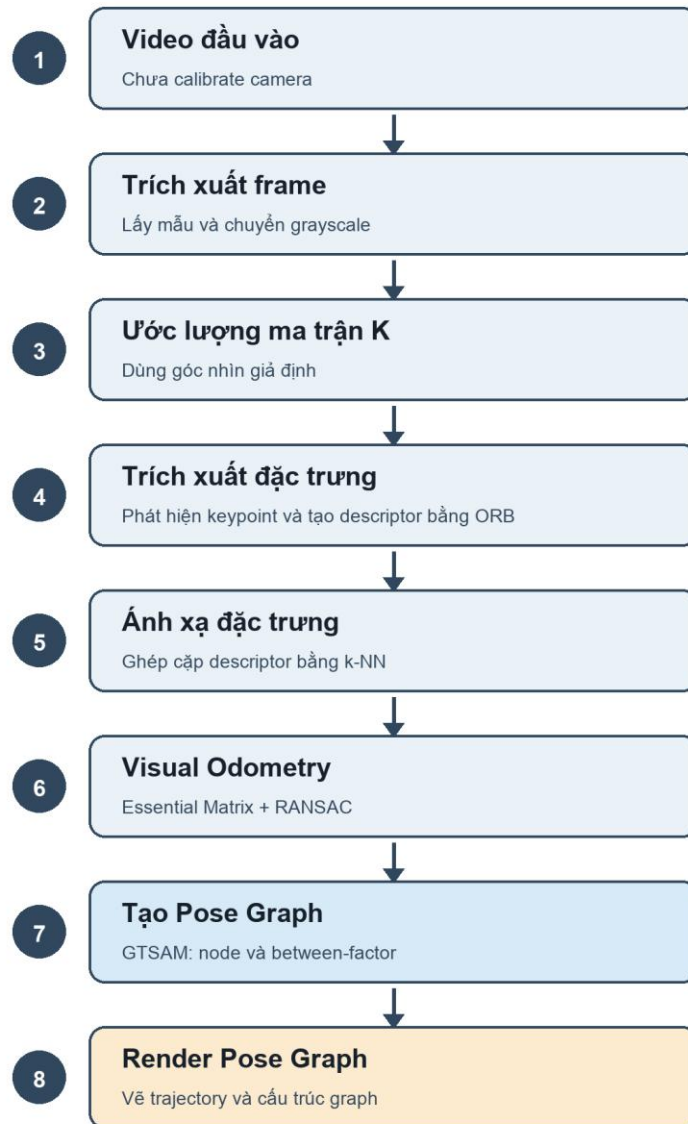
Loại SLAM	Cảm biến	Ưu điểm	Nhược điểm
Giải pháp trong đề tài	Camera đơn (camera điện thoại Iphone 13, không hiệu chỉnh)	Đơn giản, dễ tái lập; minh họa trọn vẹn front-end VO và biểu diễn Pose Graph	Không có scale metric, chưa có loop closure và bước tối ưu hoá; nhạy với rung và mờ chuyển động

Bảng 2: So sánh ba loại kiến trúc SLAM phổ biến và giải pháp trong đề tài

Từ bảng so sánh trên có thể thấy dù khác nhau về cảm biến, cả ba loại kiến trúc SLAM hoàn chỉnh đều có khối back-end tối ưu hoá và một hình thức tạo ràng buộc để bước tối ưu hoá thực sự phát huy tác dụng. Giải pháp trong đề tài chủ động giới hạn phạm vi ở front-end (VO) và biểu diễn Pose Graph, xem đây là bước nền tảng cần nắm vững trước khi mở rộng thêm các khối phía sau hướng mở rộng cụ thể được thảo luận ở mục 4.3.

3.2. Sơ đồ tổng quát giải pháp

Giải pháp được trình bày dưới dạng một pipeline gồm tám khối xử lý là đọc video, trích xuất frame, ước lượng ma trận K, trích xuất đặc trưng, so khớp đặc trưng, ước lượng Visual Odometry, dựng Pose Graph và render.



Hình 3: Kiến trúc pipeline Visual Odometry/Pose Graph được cài đặt trong đề tài

3.2.1. Khởi tiếp nhận, tiền xử lý video và ước lượng ma trận K

Khối này tiếp nhận video đầu vào và chuẩn bị dữ liệu cho toàn bộ pipeline. Video được đọc tuần tự và lấy mẫu, chỉ giữ lại một khung hình trong mỗi nhóm khung hình liên tiếp. Mỗi khung hình được đưa về một chiều rộng thống nhất giữ nguyên tỉ lệ và chuyển sang ảnh grayscale dữ liệu đầu vào của bước trích xuất đặc trưng; ảnh màu được giữ lại phục vụ trực quan hoá.

Do camera điện thoại không được hiệu chỉnh, ma trận nội tại K chưa biết. Khối tiền xử lý vì vậy ước lượng K gần đúng từ kích thước ảnh và một góc nhìn ngang giả định, với giả thiết pixel vuông và tâm ảnh trùng tâm quang học. K là đầu vào bắt buộc

của các phép hình học epipolar ước lượng Essential Matrix ở khối sau, nên sai lệch của giả định này ảnh hưởng trực tiếp tới chất lượng ước lượng chuyển động.

3.2.2. Khối trích xuất đặc trưng ORB

Khối này mô tả mỗi khung hình bằng một tập đặc trưng cục bộ, làm đầu vào cho bước so khớp phía sau. Bộ phát hiện ORB được dùng để tìm keypoint và tính binary descriptor trên ảnh grayscale; ORB được chọn vì tốc độ nhanh, không ràng buộc bản quyền và bất biến với phép xoay.

3.2.3. Khối so khớp đặc trưng bằng k-NN

Khối so khớp nhận descriptor của hai khung hình liên tiếp và ghép cặp chúng bằng BFMatcher với khoảng cách Hamming theo chiến lược k-NN ($k = 2$), mỗi descriptor ở khung hình thứ nhất được ghép với hai ứng viên gần nhất ở khung hình thứ hai. Các cặp matching sẽ được xử lý bằng Lowe ratio test chỉ giữ match khi ứng viên tốt nhất vượt trội rõ rệt so với ứng viên tốt thứ hai. Tập good match thu được là đầu vào của bước ước lượng chuyển động.

Cần phân biệt ba khái niệm trong bước này. Match là một cặp descriptor được BFMatcher ghép với nhau. Good match là match đã vượt qua Lowe ratio test. Inlier là good match tiếp tục phù hợp với Essential Matrix mà RANSAC chọn. Vì mỗi tầng dùng một điều kiện chặt hơn, số inlier luôn nhỏ hơn hoặc bằng số good match.

3.2.4. Khối Visual Odometry

Từ tập good match, chuyển động tương đối (R, t) giữa hai khung hình được ước lượng thông qua Essential Matrix trong khung RANSAC, nhờ đó các match không đồng thuận với cùng một mô hình chuyển động bị loại làm outlier.

Có thể mô tả bước ước lượng này bằng hai trường hợp. Trường hợp bình thường được dùng khi số good match đạt ngưỡng và Essential Matrix hợp lệ. Trường hợp fallback được dùng khi có ít hơn 30 good match, findEssentialMat thất bại, hoặc recoverPose không thu được inlier nào; khi đó $R = I$ và $t = 0$ coi như camera đứng yên ở bước đó. Fallback đảm bảo chuỗi pose liên mạch, mỗi khung hình một pose và chương trình tiếp tục chạy.

3.2.5. Khối xây dựng Pose Graph

Ý tưởng của khối này là chuyển chuỗi kết quả Visual Odometry thành một đồ thị, mỗi pose camera là một node, mỗi chuyển động tương đối giữa hai khung hình liên tiếp là một cạnh ràng buộc.

3.2.6. Khối render & lưu kết quả

Khối render chiếu trajectory lên mặt phẳng X-Z để quan sát hình dạng chuyển động; màu của mỗi cạnh mã hoá inlier ratio, cho phép nhận ra ngay những đoạn có độ tin cậy thấp. Ngoài hình tổng quan, một hình cận cảnh phóng to vài pose liên tiếp kèm thông số chuyển động tương đối trên từng cạnh để minh hoạ cấu trúc đồ thị.

3.3. Thư viện sử dụng và môi trường cài đặt

3.3.1. Thư viện sử dụng

Toàn bộ pipeline được cài đặt bằng Python 3.12 và chạy trong Jupyter Notebook. Kèm theo là một ứng dụng demo dạng desktop app.py dùng chung thuật toán với notebook, cho phép upload video, điều chỉnh tham số và xem kết quả từng bước trên giao diện. Bảng 3 liệt kê các thư viện chính, phiên bản được ghim và vai trò của từng thư viện trong pipeline.

Thư viện	Phiên bản	Vai trò trong pipeline
Python	3.12.5	Ngôn ngữ cài đặt toàn bộ pipeline, chạy trong Jupyter Notebook
NumPy	1.26.4	Tính toán mảng và đại số tuyến tính, ma trận xoay R, vector tịnh tiến t, ma trận nội tại K
OpenCV (opencv-contrib-python)	4.10.0.84	Đọc video, phát hiện và mô tả đặc trưng ORB, so khớp BFMatcher,

		findEssentialMat + RANSAC, recoverPose, vẽ minh hoạ match
GTSAM	4.2.1	Dựng Pose Graph, NonlinearFactorGraph, Pose3, PriorFactorPose3, BetweenFactorPose3 (pipeline không gọi optimizer)
NetworkX	3.6.1	Biểu diễn Pose Graph dạng DiGraph song song với GTSAM để render và xuất kết quả
Matplotlib	3.11.0	Vẽ khung hình mẫu, match ORB, quỹ đạo và các hình trong báo cáo
Streamlit	1.59.2	Framework giao diện của ứng dụng demo bao gồm upload video, điều chỉnh tham số, hiển thị kết quả từng bước theo tab và nút tải file đầu ra
pywebview	6.2.1	Mở ứng dụng demo trong cửa sổ desktop native trên macOS (python app.py) thay vì tab trình duyệt

Bảng 3: Danh sách thư viện sử dụng trong pipeline

3.3.2. Môi trường cài đặt

Thực nghiệm được chạy trên macOS chip Apple Silicon với Python 3.12.5. Toàn bộ thư viện được cài trong một môi trường ảo venv riêng, tách khỏi Python hệ thống, để tránh xung đột phiên bản và bảo đảm khả năng tái lập. Các bước thiết lập môi trường:

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install "numpy==1.26.4" "opencv-contrib-python==4.10.0.84"
"gtsam==4.2.1" matplotlib networkx tqdm streamlit pywebview
```

Thông tin chi tiết của lần chạy thực nghiệm với bộ tham số CONFIG được tổng hợp trong bảng 4.

3.4. Thực nghiệm

3.4.1. Môi trường và thiết lập thực nghiệm

Nhóm dùng video data/input.mp4 được quay bằng camera điện thoại trong điều kiện thực tế. Video có 2112 frame, tốc độ 60 fps, dài khoảng 35,2 giây và có chiều dọc. Đây là video tự quay, mục tiêu của lần thử là kiểm tra pipeline có chạy được end-to-end với dữ liệu thực tế hay không, chưa phải đo độ chính xác tuyệt đối của trajectory.

Lần chạy được kiểm tra theo đúng thứ tự cell của notebook cài thư viện, nạp cấu hình, đọc và lấy mẫu video, tạo ảnh grayscale và K, tính ORB cho mọi frame, chạy Visual Odometry trên 422 cặp, dựng GTSAM factor graph và NetworkX graph, render cuối cùng lưu output. Nhóm ghi lại số frame, kích thước ảnh, K, match ở cặp đầu, mean inlier ratio, số node/factor và danh sách file đầu ra.

Thông số	Giá trị
Video đầu vào	data/input.mp4
Bước lấy mẫu khung hình (FRAME_STEP)	5
Độ rộng resize (RESIZE_WIDTH)	640 px

Thông số	Giá trị
Số đặc trưng ORB tối đa (N_FEATURES)	2000
Ngưỡng tỉ lệ Lowe (LOWE_RATIO)	0,75
Ngưỡng match tối thiểu (MIN_MATCH_COUNT)	30
Ngưỡng RANSAC (RANSAC_THRESH_PX)	1,0 px
Độ tin cậy RANSAC (RANSAC_CONFIDENCE)	0,999

Bảng 4: Thông số cấu hình thực nghiệm

3.4.2. Quá trình thực hiện qua từng bước

Phần này bám theo sơ đồ tổng quát ở mục 3.2 và minh hoạ cách chương trình xử lý dữ liệu thực tế qua từng bước. Bước 1 dưới đây tương ứng ba khối đầu của hình 3 (video đầu vào, trích xuất frame và ước lượng ma trận K), các Bước 2–6 lần lượt tương ứng các khối 4–8.

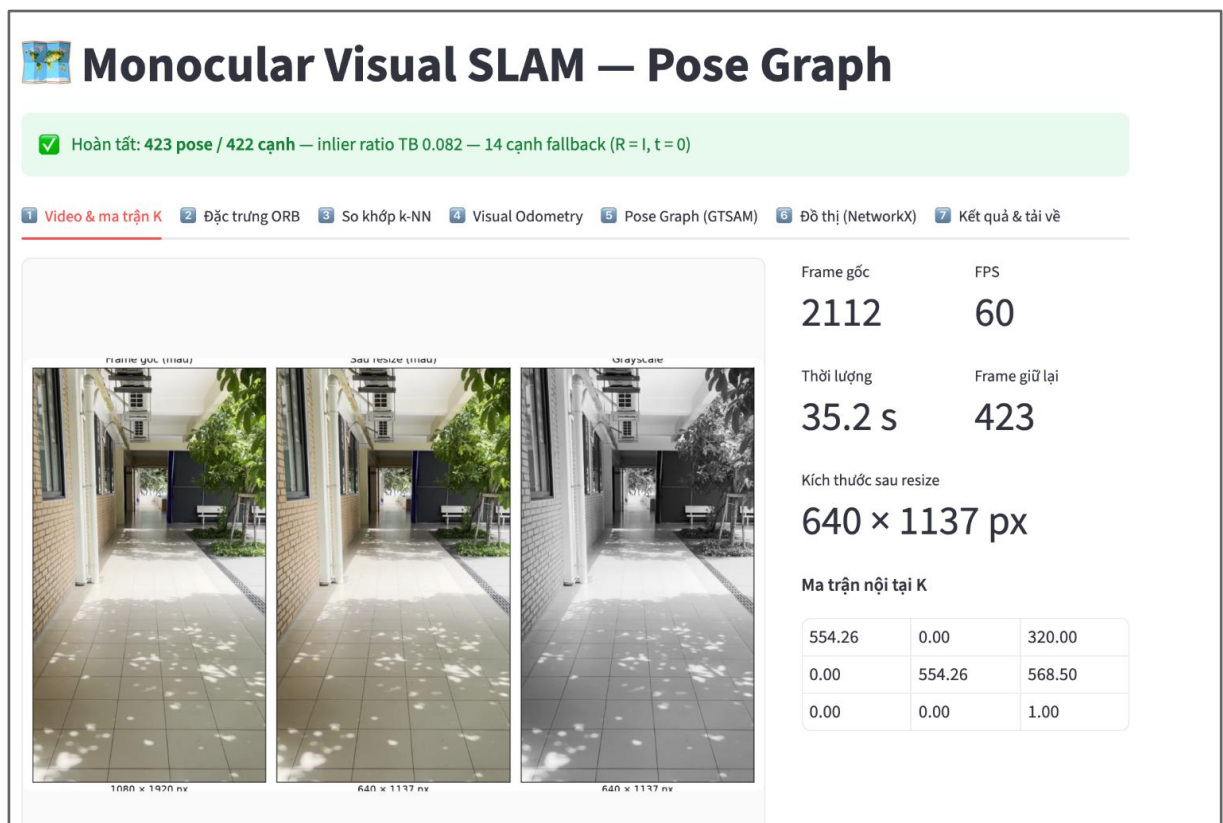
Bước 1: Tiếp nhận video, trích xuất frame và ước lượng ma trận K.

Dữ liệu đầu vào là video data/input.mp4 quay bằng camera điện thoại ở tư thế dọc, độ phân giải 1080×1920 px, tốc độ 60 fps, dài khoảng 35 giây với tổng cộng 2112 frame. Video chưa qua hiệu chỉnh camera và không kèm thông số nội tại, đúng với giả định đã nêu ở mục 3.2.1.

Video được mở bằng cv2.VideoCapture để đọc metadata (tổng số frame, fps) rồi duyệt tuần tự toàn bộ khung hình. Chương trình lấy mẫu theo chu kỳ cố định, trong mỗi 5 frame liên tiếp chỉ giữ lại một, tức chỉ giữ những frame có chỉ số chia hết cho 5. Cách lấy mẫu này giảm số ảnh cần xử lý nhưng vẫn giữ đúng thứ tự thời gian.

Mỗi frame được resize về chiều rộng 640 px bằng `cv2.resize`, giữ nguyên tỉ lệ khung hình, nên video dọc sau xử lý có kích thước khoảng 640×1137 px. Ảnh màu được giữ lại để trực quan hóa, đồng thời mỗi frame được chuyển sang ảnh mức xám bằng `cv2.cvtColor` làm đầu vào cho bước trích xuất đặc trưng. Ảnh grayscale chỉ còn một kênh cường độ, phù hợp với cách ORB tìm corner và tạo binary descriptor.

Do video chưa có thông số calibrate camera, ma trận nội tại K được ước lượng gần đúng từ kích thước ảnh và góc nhìn ngang giả định 60° tiêu cự được tính theo pixel được suy từ góc nhìn này, pixel được giả định vuông nên $f_y = f_x$, còn principal point nằm ở tâm ảnh. Đây là giá trị giả định giúp pipeline có đủ dữ liệu để chạy Essential Matrix.



Hình 4: Khung hình gốc, sau resize và ảnh grayscale (kèm kích thước từng bước)

Bước 2: Trích xuất đặc trưng ORB.

Bộ phát hiện đặc trưng được khởi tạo bằng `cv2.ORB_create` với giới hạn tối đa 2000 đặc trưng mỗi ảnh. Với từng frame, `orb.detectAndCompute` trả về tập keypoint và binary descriptor tương ứng. Keypoint/descriptor của toàn bộ 423 frame được tính trước một lần rồi lưu lại, thay vì tính lại mỗi khi xử lý một cặp frame.

 **Monocular Visual SLAM — Pose Graph**

✓ Hoàn tất: 423 pose / 422 cạnh — inlier ratio TB 0.082 — 14 cạnh fallback ($R = I$, $t = 0$)

1 Video & ma trận K

2 **Đặc trưng ORB**

3 So khớp k-NN

4 Visual Odometry

5 Pose Graph (GTSAM)

6 Đồ thị (NetworkX)

7 Kết quả & tải về

Top 150/2000 keypoint ORB theo response (frame 0)



Keypoint frame 0

2000

Keypoint TB mỗi frame

1999

Giới hạn tối đa

2000

Hình 5: Keypoint ORB trên khung hình đầu tiên

Bước 3: So khớp đặc trưng bằng k-NN.

So khớp đặc trưng dùng `cv2.BFMatcher` với khoảng cách Hamming. Với mỗi descriptor ở frame thứ nhất, hàm `knnMatch` ($k = 2$) trả về hai ứng viên gần nhất ở frame thứ hai; match chỉ được giữ khi khoảng cách của ứng viên tốt nhất nhỏ hơn 0,75 lần ứng viên thứ hai (Lowe ratio test).

Kết quả là danh sách good match cùng hai tập tọa độ điểm tương ứng trên hai frame, được dùng trực tiếp cho bước ước lượng chuyển động.

 **Monocular Visual SLAM — Pose Graph**

✅ Hoàn tất: 423 pose / 422 cạnh — inlier ratio TB 0.082 — 14 cạnh fallback ($R = I, t = 0$)

1 Video & ma trận K

2 Đặc trưng ORB

3 So khớp k-NN

4 Visual Odometry

5 Pose Graph (GTSAM)

6 Đồ thị (NetworkX)

7 Kết quả & tải về

Match k-NN (k = 2) + Lowe ratio (frame 0 → 1): 1358 good match (vẽ 60)



Cặp frame mẫu

0 → 1

Good match

1358

Lowe ratio

0.75

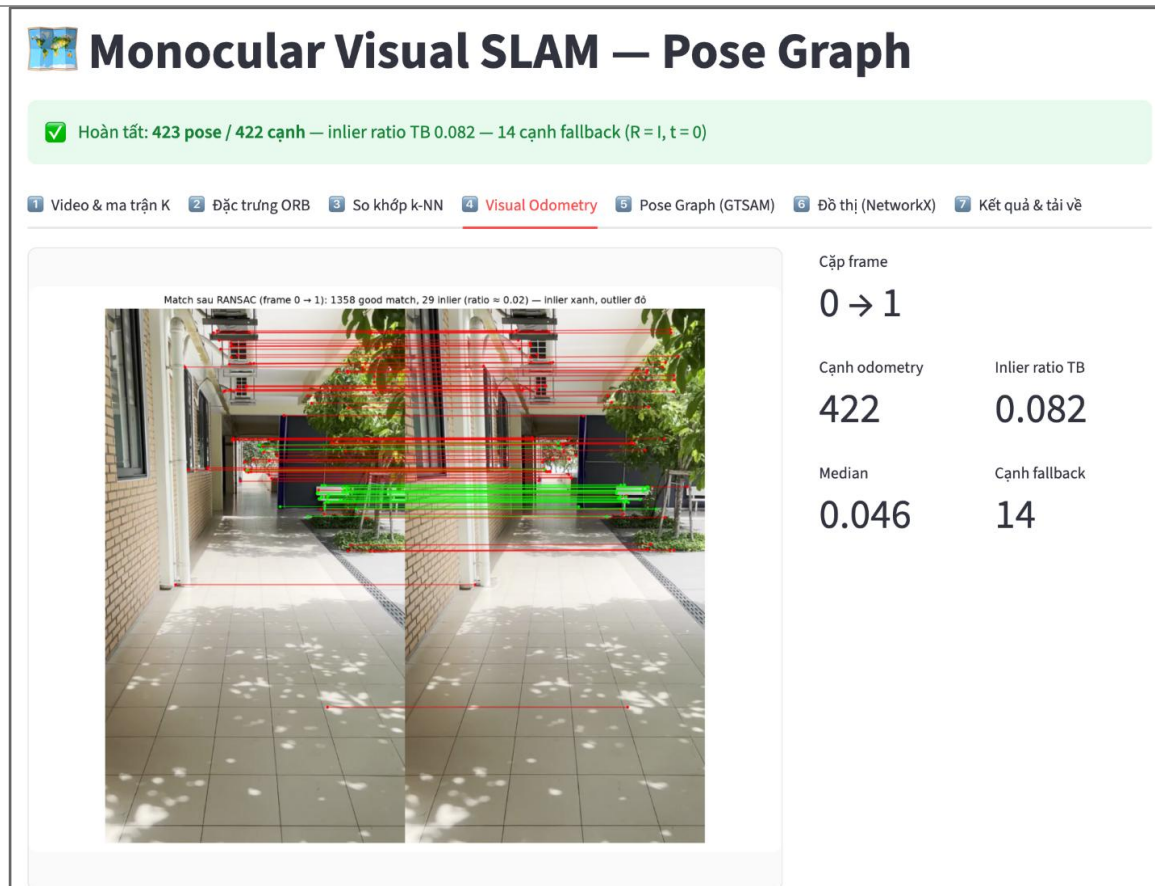
k-NN

k = 2

Hình 6: Giao diện Bước 3: mẫu good match k-NN sau Lowe ratio test giữa hai khung hình đầu chuỗi

Bước 4: Visual Odometry ước lượng chuyển động tương đối và nối chuỗi pose.

Nếu một cặp frame có ít nhất 30 good match, `cv2.findEssentialMat` được gọi trên hai tập tọa độ điểm tương ứng, với ngưỡng sai số RANSAC 1 pixel và độ tin cậy 0,999. `cv2.recoverPose` tiếp tục phân rã Essential Matrix để lấy rotation R , translation t và số inlier. Inlier ratio được tính bằng số inlier chia cho số điểm match đầu vào, sau đó dùng làm chỉ báo độ tin cậy cho cạnh odometry tương ứng.



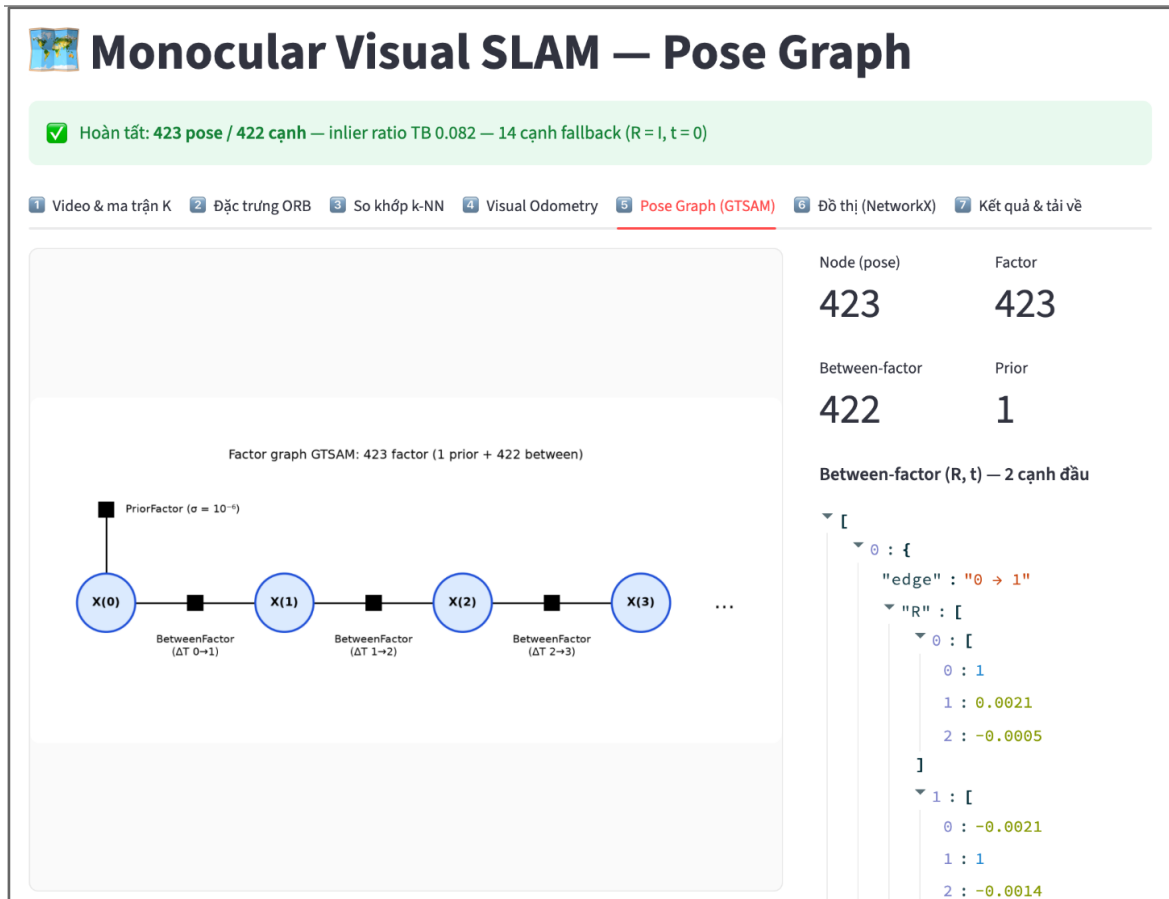
Hình 7: Thống kê Visual Odometry và match sau RANSAC ở cặp inlier (xanh), outlier (đỏ)

Bước 5: Dựng Pose Graph.

Mỗi cặp R, t do Visual Odometry trả về được chuyển thành một đối tượng `gtsam.Pose3`, gồm `gtsam.Rot3` cho phần xoay và `gtsam.Point3` cho phần tịnh tiến.

Chương trình tạo hai đối tượng chính một `gtsam.NonlinearFactorGraph` chứa các factor và một `gtsam.Values` chứa giá trị pose ban đầu. Pose đầu tiên $X(0)$ được neo tại gốc bằng `PriorFactorPose3` với $\sigma = 10^{-6}$ cho cả sáu chiều, giúp hệ tọa độ có một mốc cố định.

Với mỗi cặp frame $i \rightarrow i+1$, chuyển động tương đối được tính bằng phương thức `between` của `gtsam.Pose3` rồi đưa vào một `BetweenFactorPose3`. Mô hình nhiễu của factor là `gtsam.noiseModel.Diagonal` với σ tỉ lệ nghịch với inlier ratio của cạnh, cặp frame càng ít inlier thì σ càng lớn, phản ánh độ tin cậy thấp hơn của phép đo. Đồng thời pose $X(i+1)$ được thêm vào tập giá trị khởi tạo. Sau vòng lặp, factor graph có 1 prior và 422 between-factor, còn tập khởi tạo có 423 `Pose3`.



Hình 8: Giao diện tab Pose Graph (GTSAM) số node/factor, sơ đồ factor graph với bốn pose đầu $X(0)$ – $X(3)$ và BetweenFactor giữa các pose kế tiếp.

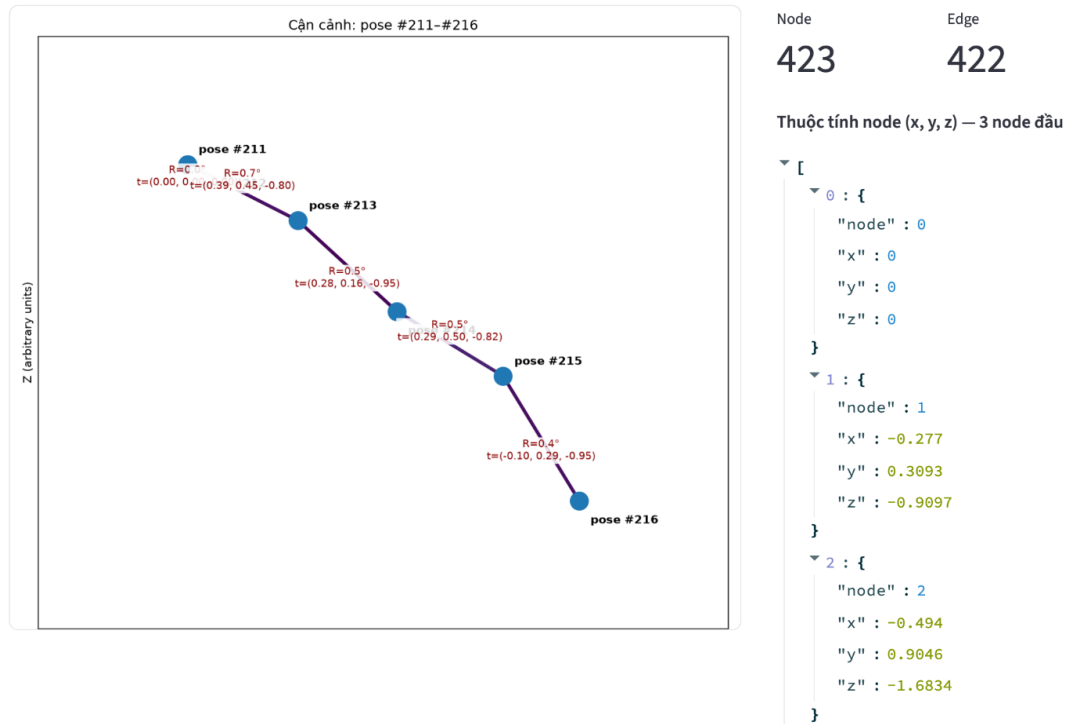
Song song với GTSAM, chương trình dựng một đồ thị có hướng networkx. DiGraph. Mỗi node lưu tọa độ x, y, z của pose, mỗi cạnh lưu loại ràng buộc, số inlier và inlier ratio. NetworkX không thay GTSAM trong bài toán factor graph nó được dùng vì thuận tiện cho việc xem cấu trúc, tô màu cạnh và lưu bằng Pickle. Hình số 9 bên dưới phóng to các node liên tiếp, kèm góc rotation và hướng translation của từng cạnh.



Monocular Visual SLAM — Pose Graph

✓ Hoàn tất: 423 pose / 422 cạnh — inlier ratio TB 0.082 — 14 cạnh fallback ($R = I, t = 0$)

1 Video & ma trận K 2 Đặc trưng ORB 3 So khớp k-NN 4 Visual Odometry 5 Pose Graph (GTSAM) 6 Đồ thị (NetworkX) 7 Kết quả & tải về



Hình 9: Giao diện đồ thị (NetworkX) cận cảnh cấu trúc Pose Graph thực tế, mỗi cạnh kèm chuyển động tương đối (R, t)

Bước 6: Render và lưu kết quả.

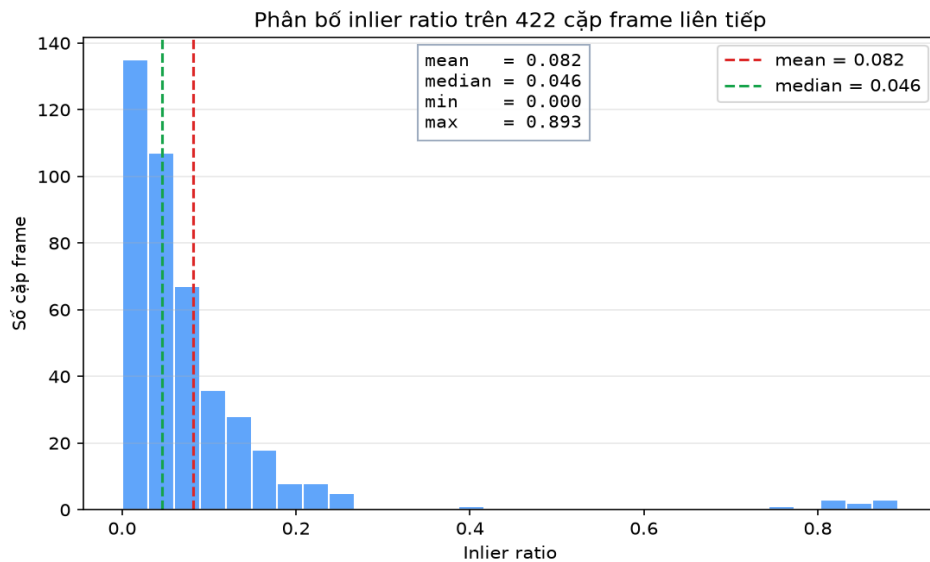
Bước render lấy tọa độ X và Z của mỗi node làm mặt phẳng hiển thị. Hình tổng quan vẽ toàn bộ 423 node và 422 cạnh; màu mỗi cạnh mã hoá inlier ratio theo colormap viridis. Dấu sao màu xanh đánh dấu pose đầu tiên. Hình cận cảnh chọn sáu pose ở giữa trajectory, hiển thị chỉ số pose, góc rotation và ba thành phần translation trên từng cạnh.

3.4.3. Kết quả thực nghiệm

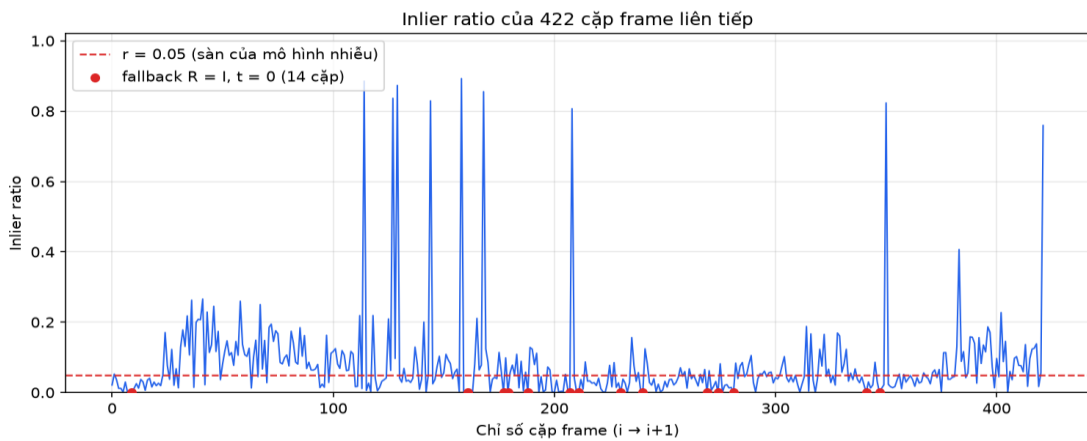
Sau khi lấy 1/5 số frame, pipeline giữ lại 423 frame ở kích thước khoảng 640×1137 px. Ma trận K giả định có $f_x = f_y \approx 554,26$; $c_x = 320$; $c_y = 568,5$. ORB được tính trước cho toàn bộ frame. Ở cặp frame $0 \rightarrow 1$, Lowe ratio test giữ lại 1358 good match. Đây là số match trước bước RANSAC, vì vậy không nên hiểu cả 1358 điểm đều là inlier của cùng một Essential Matrix.

Visual Odometry tạo được kết quả cho cả 422 cặp frame liên tiếp, từ đó hình thành 423 pose. Pose Graph cuối cùng có 423 node và 423 factor. Điều này xác nhận rằng dữ liệu đã đi hết pipeline và không bị đứt giữa chừng. Tuy nhiên, mean inlier ratio chỉ khoảng 0,08, nghĩa là trung bình chỉ khoảng 8% match được RANSAC xem là phù hợp với cùng một mô hình chuyển động. Đây là dấu hiệu cho thấy chất lượng ước lượng R, t chưa thật sự ổn định.

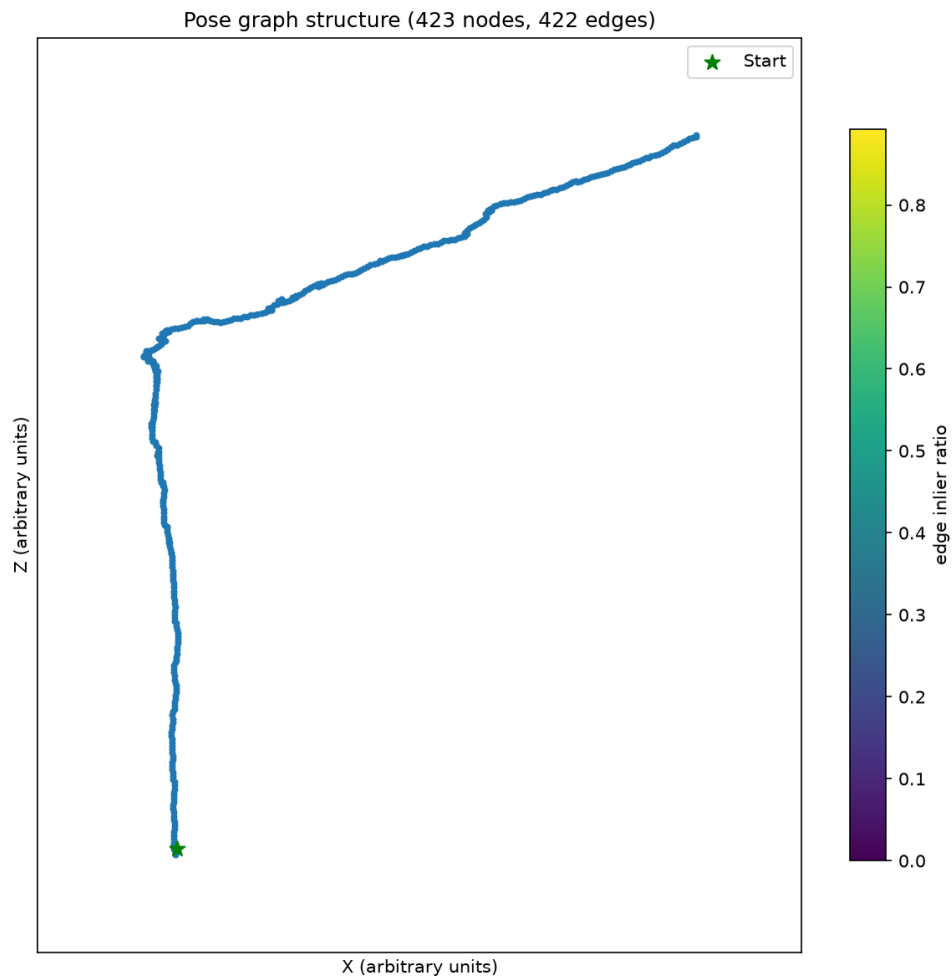
Phân bố chi tiết của inlier ratio được thể hiện ở Hình 10 và Hình 11 Trên 422 cặp khung hình, median là 0,046, giá trị nhỏ nhất 0,000 và lớn nhất 0,893; có 14 cặp rơi vào nhánh fallback ($R = I$, $t = 0$). Hình 10 cho thấy giá trị thấp không tập trung ở một đoạn duy nhất mà rải rác trên toàn video.



Hình 10: Phân bố inlier ratio trên 422 cặp khung hình liên tiếp



Hình 11: Inlier ratio theo từng cặp khung hình; các điểm đỏ là cặp rơi vào nhánh fallback



Hình 12: Pose Graph thực tế sau khi dựng và render (423 node, 422 cạnh), cạnh tô màu theo inlier ratio

Chỉ số	Giá trị	Đơn vị	Ghi chú
Khung hình đã xử lý	423	khung hình	Từ 2112 khung gốc (FRAME_STEP = 5)
Match ORB (khung 0 → 1) Match ORB (khung 0 → 1)	1358	match	Vượt xa ngưỡng tối thiểu (30)
Inlier ratio trung bình	0,08	tỉ lệ (8%)	Xem phân tích ở mục 3.5.2
Node Pose Graph	423	node	1 node / khung hình

Chỉ số	Giá trị	Đơn vị	Ghi chú
Factor Pose Graph	423	factor	1 prior + 422 between-factor odometry
Back-end optimizer	Không chạy	—	Notebook chỉ dựng graph và initial

Bảng 5: Kết quả định lượng thực nghiệm

3.5. Phân tích và đánh giá kết quả

Kết quả tích cực nhất là notebook chạy hết chuỗi 423 frame. Số lượng 423 node, 423 edge mỗi frame tạo một pose, mỗi cặp liên tiếp tạo một edge. Cơ chế fallback giúp vòng lặp không dừng khi thiếu match hoặc findEssentialMat thất bại.

Kết quả cũng bộc lộ rõ các hạn chế cố hữu của một pipeline chỉ-odometry:

- Không có scale metric do scale ambiguity đơn mắt.
- Camera không hiệu chỉnh ma trận K được giả định từ một góc nhìn ngang ước lượng 60° , không đo đặc thật, nên toàn bộ hình học essential-matrix có thể bị sai lệch hệ thống nếu góc nhìn thật khác biệt đáng kể.
- Mean inlier ratio thấp ($\sim 0,08$) phần lớn cạnh odometry trong Pose Graph có độ tin cậy thấp, có khả năng cao do rung, mờ chuyển động khi quay video bằng tay, K không chính xác, hoặc cảnh có kết cấu lặp lại gây match nhầm.
- Không có loop closure/tối ưu hoá Đồ thị chỉ-odometry là một chuỗi không chu trình, nên toàn bộ drift tích lũy từ visual odometry không được hiệu chỉnh lại ở bất kỳ bước nào của pipeline.

CHƯƠNG 4: KẾT LUẬN

4.1. Kết luận

Qua đề tài, nhóm đã tìm hiểu các phần chính của SLAM, Pose Graph và Monocular Visual Odometry, gồm Essential Matrix, RANSAC và scale ambiguity. Từ phần lý thuyết này, nhóm xây dựng được một pipeline chạy trên video thật lấy frame, tìm và match feature ORB, ước lượng chuyển động camera, sau đó tạo và vẽ Pose Graph bằng GTSAM.

Trong lần chạy thực nghiệm, pipeline xử lý 423 frame, tìm được 1358 match ORB ở cặp frame đầu và tạo Pose Graph gồm 423 node, 423 factor. Chương trình chạy hết dữ liệu và xuất được các file kết quả. Tuy nhiên, mean inlier ratio còn thấp và trajectory chưa có ground truth để đối chiếu. Notebook không chạy optimizer vì graph chỉ có các factor odometry được tạo từ chính chuỗi pose ban đầu. Kết quả này cho thấy bước tiếp theo nên là calibrate camera, bổ sung loop closure và đánh giá trên dữ liệu có ground truth.

4.2. Hạn chế

- Camera không được hiệu chỉnh, nên toàn bộ hình học essential-matrix và Pose Graph có thể chịu sai lệch hệ thống nếu giả định này khác nhiều so với camera thật.
- Không có nguồn thông tin tỉ lệ nên khoảng cách trong Pose Graph chỉ mang tính tương đối, không phải đơn vị mét thật.
- Chưa triển khai loop closure và bước tối ưu hoá Pose Graph, nên drift tích lũy từ visual odometry không được hiệu chỉnh lại.
- Mean inlier ratio thực nghiệm khá thấp ($\sim 0,08$), cho thấy chất lượng ước lượng chuyển động cục bộ còn nhiều hạn chế do có thể do rung, mờ chuyển động khi quay bằng tay.
- Chưa xây dựng bản đồ điểm mốc 3D, nên chưa thể dùng kết quả để dựng lại bản đồ điểm đám mây của môi trường.
- Pipeline xử lý theo dạng batch, chưa hỗ trợ chạy trực tuyến

4.3. Hướng phát triển

- Bổ sung cơ chế loop closure để tạo ràng buộc, làm cho bước tối ưu hoá Pose Graph thực sự có giá trị sửa drift.
- Hiệu chỉnh camera thật thay cho giả định góc nhìn ngang hiện tại.
- Mở rộng factor graph với landmark 3D để có bản đồ điểm, tiến tới SLAM đầy đủ thay vì chỉ pose-graph.
- Mở rộng bài toán sang Dynamic SLAM khi front-end và back-end tĩnh đã ổn định.

TÀI LIỆU THAM KHẢO

- [1] C. C. a. L. C. a. H. C. a. Y. L. a. D. S. a. J. N. a. I. R. a. J. Leonard, "Past, Present, and Future of Simultaneous Localization And Mapping: Towards the Robust-Perception Age," *IEEE Transactions on Robotics*, vol. 32, no. 6, p. 1309–1332, 2016.
- [2] R. K. C. S. W. B. Giorgio Grisetti, "A Tutorial on Graph-Based SLAM," *IEEE Intelligent Transportation Systems Magazine*, vol. 2, no. 4, pp. 31 - 43, 2010.
- [3] R. M.-A. a. J. D. T. ́. os, "ORB-SLAM2: an Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras," *IEEE Transactions on Robotics*, vol. 33, no. 5, pp. 1255 - 1262, 2017 .
- [4] J. Z. J. L. Baichuan Huang, "A Survey of Simultaneous Localization and Mapping with an Envision in 6G Wireless Networks," *IEEE Transactions on Robotics*, 2024.
- [5] F. A. Z. A. E. Ali Ghadimzadeh Alamdari, "A Review of Simultaneous Localization and Mapping for the Robotic-Based Nondestructive Evaluation of Infrastructures," *Feature Review Papers in Intelligent Sensors*, vol. 25, no. 3, 2025.